

CS-590 Database Design and Development

5-2 Assignment: Developing a Schema Using MongoDB

Malgorzata Debska

Southern New Hampshire University

Zach Probst

March 2, 2026

Developing a Document-Based Schema in MongoDB for the Child Nutrition Database

In today's data ecosystems, organizations must carefully evaluate data models to ensure efficient storage, retrieval, scalability, and integration across systems. As a data engineer tasked with evaluating a transition from a legacy relational system to a document-oriented NoSQL solution such as MongoDB, it is essential to understand how schema design impacts performance and usability. This assignment extends the evaluation of migrating the Child Nutrition database from PostgreSQL into MongoDB by modeling a document-based schema using four additional tables: Food Category, Food Description, GPC Category, and Weights. The purpose of this task was to implement a document collection that integrates multiple data sources, perform extract-transform-load (ETL) operations, execute sentinel queries to verify integration success, validate the schema design, and create a database backup using mongodump. This process reflects real-world responsibilities in data engineering and database architecture.

Implementation of the Document Collection

In the relational PostgreSQL model, data was normalized across multiple tables and connected through foreign key relationships. To adapt this structure to MongoDB, a denormalized collection named 'foods' was created. Each document represents a single food item and embeds related category, GPC, and weight data. This approach reduces the need for joining operations and improves read efficiency for analytics dashboards.

Extract, Transform, and Load (ETL) Process

Extract

The following JSON datasets were accessed from the Codio Assignment Resources folder:

- Module_Five_CTGNME.json (Food Category Name)
- Module_Five_FDES.json (Food Description)
- Module_Five_GPCNME.json (GPC Category Name)
- Module_Five_WGHT.json (Weights)

These files represented structured relational data previously stored in PostgreSQL.

Transform

The provided JSON files contained wrapper objects (for example, "CTGNME", "FDES", "GPCNME", and "WGHT") enclosing arrays of records. MongoDB's mongoimport tool requires properly formatted JSON arrays when using the --jsonArray option. Therefore, each file was transformed by extracting only the inner arrays and saving them as new array-only JSON files. Also, field structures were evaluated to ensure that related attributes could be embedded into a single document. Numeric and textual values were preserved as stored in the source JSON files. This transformation phase ensured compatibility with MongoDB and prepared the data for schema implementation.

Load

The transformed array files were imported into MongoDB using the mongoimport command:

```
mongoimport --db child_nutrition --collection ctgnme --file ctgnme_array.json --jsonArray
```

```
mongoimport --db child_nutrition --collection fdes --file fdes_array.json --jsonArray
```

```
mongoimport --db child_nutrition --collection gpcnme --file gpcnme_array.json --jsonArray
```

```
mongoimport --db child_nutrition --collection wght --file wght_array.json --jsonArray
```

After loading, an aggregation pipeline was used to combine related data into the food collection.

This step effectively implemented the document schema.

```
ubuntu@i-03030545a0bddeb5f:~/Desktop/Assignment Resources/Module Five Assignment Developing a Schema Using MongoDB$ mongoimport
--db child_nutrition --collection ctgnme --file ctgnme_array.json --jsonArray
2026-03-02T22:01:17.730+0000    connected to: mongodb://localhost/
2026-03-02T22:01:17.753+0000    25 document(s) imported successfully. 0 document(s) failed to import.
ubuntu@i-03030545a0bddeb5f:~/Desktop/Assignment Resources/Module Five Assignment Developing a Schema Using MongoDB$ mongoimport
--db child_nutrition --collection fdes --file fdes_array.json --jsonArray
2026-03-02T22:02:15.149+0000    connected to: mongodb://localhost/
2026-03-02T22:02:15.612+0000    8976 document(s) imported successfully. 0 document(s) failed to import.
ubuntu@i-03030545a0bddeb5f:~/Desktop/Assignment Resources/Module Five Assignment Developing a Schema Using MongoDB$ mongoimport
--db child_nutrition --collection gpcnme --file gpcnme_array.json --jsonArray
2026-03-02T22:02:49.101+0000    connected to: mongodb://localhost/
2026-03-02T22:02:49.128+0000    131 document(s) imported successfully. 0 document(s) failed to import.
ubuntu@i-03030545a0bddeb5f:~/Desktop/Assignment Resources/Module Five Assignment Developing a Schema Using MongoDB$ mongoimport
--db child_nutrition --collection wght --file wght_array.json --jsonArray
2026-03-02T22:03:16.003+0000    connected to: mongodb://localhost/
ubuntu@i-03030545a0bddeb5f:~/Desktop/Assignment Resources/Module Five Assignment Developing a Schema Using MongoDB$
```

Aggregation and Schema Integration

An aggregation pipeline using \$lookup, \$project, and \$out was executed within mongosh to simulate relational joins and produce a fully integrated 'foods' collection. The \$lookup stages joined category, GPC, and weight data based on matching keys, while \$project structured the final document layout. The \$out stage saved the transformed results into the food collection.

Applications: Terminal - mongosh mo...

Terminal - mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=: ^ _ □ ×

File Edit View Terminal Tabs Help

2026-03-02T22:14:29.031+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See <http://dochub.mongodb.org/core/prodnotes-filesystem>

2026-03-02T22:14:32.805+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted

test> use child_nutrition

switched to db child_nutrition

child_nutrition> show collections

ctgnme

fdes

gpcnme

nutrient_values

wght

child_nutrition> db.ctgnme.countDocuments()

25

child_nutrition> db.fdes.countDocuments()

8976

child_nutrition> db.gpcnme.countDocuments()

131

child_nutrition> db.wght.countDocuments()

15052

child_nutrition>

```
Terminal - mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
File Edit View Terminal Tabs Help
ubuntu@i-03030545a0bdeb5f:~$ mongosh
Current Mongosh Log ID: 69a60dalf47b0d2f478de665
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:      6.0.27
Using Mongosh:       2.5.10
mongosh 2.7.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-03-02T22:21:56.177+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-03-02T22:22:00.030+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use child_nutrition
switched to db child_nutrition
child_nutrition> db.fdes.aggregate([{$lookup: {from:"ctgnme",localField:"row.Food category code", foreignField: "row.Food category code", as:"category"}},{ $lookup: {from:"Gpcnme",localField:"row.Gpc code", foreignField:"row.Gpc code", as: "gpc"}},{ $lookup: {from:"wght",localField:"row.Cn code", foreignField:"row.Cn code", as: "weights"}}, {$project:{food_code:"$row.Cn code", description: "$row.Descriptor", category:{$arrayElemAt: ["$category.row",0]},gpc_category: {$arrayElemAt: ["$gpc.row", 0]}, weights: "$weights.row"}},{out:"foods"}}]

child_nutrition>
child_nutrition> |
```

Sentinel Query Validation

Sentinel queries were executed to confirm successful integration. The show collections command confirmed that all expected collections were present. Document count queries verified the correct record totals across collections. Additionally, querying db.foods.countDocuments() confirmed that 8,976 integrated food documents were created. A sample query using db.foods.findOne() showed that embedded category and related data were correctly structured within each document.

```
child_nutrition> db.foods.countDocuments()
8976
child_nutrition> db.foods.findOne()
{
  _id: ObjectId('69a608e725510bb5d421a38f'),
  food_code: '1001',
  description: 'Butter, salted',
  category: {
    'Food category code': '1',
    'Category description': 'Dairy and Egg Products',
    'Date added': '10/29/1997',
    'Last modified': '04/09/2012'
```

```
Terminal - mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
File Edit View Terminal Tabs Help
ubuntu@i-03030545a0bddeb5f:~$ mongosh
Current Mongosh Log ID: 69a60d1f47b0d2f478de665
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.10
Using MongoDB:  6.0.27
Using Mongosh:  2.5.10
mongosh 2.7.0 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-03-02T22:21:56.177+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-03-02T22:22:00.030+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use child_nutrition
switched to db child_nutrition
child_nutrition> db.fdes.aggregate([{$lookup: {$from:"ctgnme",localField:"row.Foochild_nutrition",foreignField:"row.Cn code",as:"category"},{$lookup: {$from:"Gpcnme",localField:"row.Gpc code",foreignField:"row.Gpc code",as:"gpc"},{$lookup: {$from:"wght",localField:"row.Cn code",foreignField:"row.Cn code",as:"weights"}},{$project:{$food_code:"$row.Cn code",description:"$row.Descriptor",category:{$arrayElemAt: ["$category.row",0]},gpc_category: {$arrayElemAt: ["$gpc.row", 0]}, weights: "$weights.row"}},{out:"foods"}}])

child_nutrition>
child_nutrition> █
```

Database Backup Using mongodump

To ensure data preservation and meet submission requirements, a backup of the instantiated MongoDB database was created using the mongodump tool:

```
mongodump --db child_nutrition --out module5-2-Debska
```

This command generated BSON and metadata files standing for the full database state. The backup folder was uploaded to the private GitHub repository under the Module Five branch, titled module5-2-Debska, as needed.

```
child_nutrition> exit
ubuntu@i-03030545a0bddeb5f:~$ mongodump --db child_nutrition --out module5-2-Debska
2026-03-02T22:49:33.129+0000    writing child_nutrition.nutrient_values to module5-2-Debska/child_nutrition/nutrient_values.bson
2026-03-02T22:49:33.130+0000    writing child_nutrition.wght to module5-2-Debska/child_nutrition/wght.bson
2026-03-02T22:49:33.130+0000    writing child_nutrition.foods to module5-2-Debska/child_nutrition/foods.bson
2026-03-02T22:49:33.130+0000    writing child_nutrition.fdes to module5-2-Debska/child_nutrition/fdes.bson
2026-03-02T22:49:33.216+0000    done dumping child_nutrition.wght (15052 documents)
2026-03-02T22:49:33.216+0000    writing child_nutrition.gpcnme to module5-2-Debska/child_nutrition/gpcnme.bson
2026-03-02T22:49:33.228+0000    done dumping child_nutrition.gpcnme (131 documents)
2026-03-02T22:49:33.228+0000    writing child_nutrition.ctgnme to module5-2-Debska/child_nutrition/ctgnme.bson
2026-03-02T22:49:33.233+0000    done dumping child_nutrition.ctgnme (25 documents)
2026-03-02T22:49:33.293+0000    done dumping child_nutrition.foods (8976 documents)
```

 GosiaD37 / CS590

<> Code

Issues

Pull requests 1

Actions

Projects

Security

Insights

Settings

 **CS590** Private

Watch 0 Fork

Module-5 11 Branches 0 Tags

Go to file

Add file

<> Code

This branch is 3 commits ahead of and 13 commits behind main .

Contribute

 GosiaD37 Add files via upload eb6cc93 · 6 minutes ago 6 Commits

 module5-2-Debska/child_nutrition	Add files via upload	6 minutes ago
 README.md	Update README with repository details	last month
 ctgnme.bson	Add files via upload	20 minutes ago

About

No description provided

Readme

Activity

0 stars

0 watchers

0 forks

Evaluation of Document-Based Schema

The migration proved that MongoDB offers flexibility in schema design and improved read performance through document embedding. This structure is helpful for analytics dashboards

requiring rapid data retrieval. However, relational databases such as PostgreSQL provide stronger schema enforcement, normalization, and transactional integrity. The choice between systems depends on whether flexibility and scalability or strict relational consistency is the organizational priority.

Conclusion

This assignment provided practical experience designing a document-based schema in MongoDB to support real-time analytics. By implementing a document collection, performing ETL operations, executing sentinel queries, confirming schema integrity, and creating a database backup, the feasibility of migrating the Child Nutrition database was evaluated. The document-oriented model supported intended queries and proved improved retrieval efficiency for embedded data structures. However, the choice between relational and document databases ultimately depends on organizational priorities, including data integrity requirements, scalability needs, and system flexibility. For structured government data, PostgreSQL may remain the best long-term solution, while MongoDB offers significant advantages for analytics-driven and evolving datasets.